

TOPICS INCLUDED

- Pitfalls in relational database designs, Concept of normalization, Function Dependencies, FD closure, First Normal Form, 2NF, 3NF, BCNF, 4NF.

4.1 PITFALLS IN RELATIONAL DATABASE DESIGNS

1. Poor Design/Planning

- The database is a vital aspect of every custom software, hence taking the time to map out the goals of database design ensures the success of any project. Consequences of lack of planning are seen further down the line and impacts projects in terms of time management.
- Improper planning of the database leaves you with no time to go back and fix errors and leads to malicious cyber attacks. Therefore, consider sitting down with a paper and drawing a data model as per business requirements.
- **For example**, coders use database schemas to incorporate database designs as they are the blueprints that help developers to visualize databases. Developers can avoid poor planning/design by checking off the following points.
 - Main tables of your database model,
 - Names for tables,
 - Rules for naming tables,
 - Time span required for the project.
- These pointers help resolve essential issues within a project and skipping any of these will only delay your project.

2. Ignoring Normalization

- Normalization or SQL (Structured Query Language) groups data under a single table and indirectly related data are put under separate tables. These tables are connected with a logical relationship between child and parent tables.
- Lack of normalization reduces data consistency and leads to duplication of data because entire data is not stored in one place. Finding related data is strenuous due to lack of grouping and costs time for searching. Hence, consider implementing normalization rules during database design.
- Despite following normalization rules, databases don't function as required. That's because they need to be

normalized to a third normal form, as this layout represents entities and is balanced with updating - inserting - deleting records.

- **For example**, don't comply with 1NF, 2NF, or 3NF and redesign the entire table.

3. Redundant Records

- Redundancy in a database is a condition in which the same set of data is stored at two different places. This means two different spots or two separate fields in software. This is a problem for developers because they have to keep several versions of data updated.
- Redundant records lead to unnecessary growth in database size, which in turn decreases the efficiency of the database and causes data corruption. Hence, the best option is to avoid preparing redundant records, unless it is essential for backups.
- Data redundancy is classified into two aspects - wasteful and excessive. Out of these, wasteful redundancy occurs when a set of data is repeated needlessly. Complicated data storing or inefficient coding results in wasteful redundancy.
- **For example**, a table of students' attributes includes a student name, student ID, college name, and course applied. Out of these, the values of college name and course are repeated which can cause issues such as insertion anomaly, update anomaly, and deletion anomaly.
- To avoid redundant records, always delete data that is no longer needed.

4. Poor Naming Standards

- Naming is a personal choice, however, it is an important aspect of documentation. Poor naming standards result in messy and large data files, hence consider incorporating consistency.
- The purpose of naming is to allow all future developers or programmers to easily understand the components of databases and what was their use. This saves time for developers and they need not go through documents to understand the meaning of a name.
- There is not a universal guide to naming conventions, but it is best to avoid bad naming practices. Here are examples of unsuccessful naming conventions that one must avoid.

(a) Underscore for Word Separation

- For certain display situations, an underscore is confused with a blank space in command and a developer may confuse it for a compound name with two objects.
- Underscore is a combination of keys, and on keyboards that are configured for international usage, the key combination is difficult to locate.
- Finally, an underscore makes a name longer, with a character that holds no meaning.

(b) Meaningless or Generic Names

- Names such as flags, scrap, data table, or config are ambiguous and misleading. Generic or meaningless names are a problem for new developers as they have to use and read data models all over again.
- Hence, for names use infinitive verbs that express actions and not static names, for example, calculate, summarize, or append.

(c) ALL UPPER CASE

- All-caps don't allow you to use camel case and make words stick together or forces you to use underscore to separate words. This impacts your reading as well cause it becomes hard to distinguish hundreds of files in one go.
- To avoid bad naming, pick one standard and stick to it; ensure that names make sense to other developers.

3. Lack of Documentation

- As per a survey conducted, the second most challenging task faced by developers was poor technical documentation. Lack of documentation leads to the loss of vital information or a tedious handover process to a new programmer.
- Consider documenting everything you know from day one because any documentation is better than none. Well-organized documentation throughout the project helps to wrap up everything smoothly and in turn, helps build robust software.
- The goal of documentation is to provide information to the support programmer to detect bugs and fix them. Documentation starts with naming columns, objects, and tables in a database model. A well-documented data model consists of solid names, definitions on columns, tables, relationships, and check and default constraints.
- It is recommended to use sample values and everything else that is to be known for a year-long project.

6. One Table to Hold All Domain Values

- The next common pitfall encountered while designing a database is to prepare one table for all the same values. For example, you have a range of values for varied areas such as order status, account status, and payment status; each one of them with different values.
- The first thought that comes to mind is to store all the values in one place because they are all status values. So, a table would look like:
 - Table or entity (order, account, and payment)
 - Key (1,2, or 3 for whole table/entity)
 - Value (pending, draft, paid etc.)
- This table looks simple, yet it comes with its own set of issues. This approach does not include referential integrity because there's no simplified way to assure statuses that are applicable to a table are associated with that specific table.
- For example, you can not relate the primary key to the account table and make sure that only account statuses are chosen.
- Hence, to avoid these hassles, consider creating different tables for all similar data values, for instance, `order_status`, `account_status` etc.

7. Ignoring Frequency or Purpose of the Data

- By ignoring the fundamental purpose of data, a designer shifts away from the primary goal of storing and retrieving data efficiently when needed. Lack of purpose of data results in no knowledge of what the data represents, at what rate and how it is going to be acquired, what will be the operational volume, and how to use it.
- For instance, a system where data is collected each day manually will not have the same data model where information is created in real-time. That is because of managing a few thousand of data monthly is different as compared to handling millions of them in the same time period.
- Further, data volume is not the only facet to consider because the purpose of data impacts data structure, normalization, implementation, and record size of the entire system.
- Clarity of purpose helps to develop database designs, record size, create entities, choose database engine management policies, and formats. By ignoring these, designs will be flawed fundamentally, even though they are mathematically and structurally right.

8. Insufficient Indexing

- Insufficient indexing comes from a SQL configuration whose performance is affected due to improper,

excessive, or missing indexes. In case indexes are not created properly, the SQL server goes through more records to retrieve the data that's requested by the query.

- A wrong index does not offer easy data manipulation and an index developed on multiple columns slows down queries instead of speeding them up. The lack of a clustered index in a table is a form of poor indexing. Execution of inserting, SELECT statement, deleting, and updating records is slower than on a clustered index.
- Index efficiency is connected to the column type, for instance, indexes on INT column display the best performance, however, indexes on DATE, VARCHAR, or DECIMAL are not as efficient. This leads to redesigning tables with the best possible efficiency.
- Overall, indexing is a complex decision because too much indexing is bad as little indexing, as it impacts the final outcome.

9. Lack of Testing

- The lack of database testing fails to give information on whether the data values stored and received in the database are valid or not. Testing helps to save transaction data, avoids data loss, and prevents unauthorized access to information.
- The database is essential for every type of software application, therefore testers need to know about SQL during testing. Consider testing for a banking application, and during tests a few things to note are:
 1. No loss of information during the process.
 2. Application stores transaction data correctly in the database and displays it accurately.
 3. No aborted or partial operation data is saved by the application.

4.2 CONCEPT OF NORMALIZATION

- Normalization is an important process in database design that helps improve the database's efficiency, consistency, and accuracy. It makes it easier to manage and maintain the data and ensures that the database is adaptable to changing business needs.
- Database normalization is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data but at different places).
- Data redundancy unnecessarily increases the size of the database as the same data is repeated in many places. Inconsistency problems also arise during insert, delete, and update operations.
- In the relational model, there exist standard methods to quantify how efficient a databases is. These

methods are called normal forms and there are algorithms to convert a given database into normal forms.

- Normalization generally involves splitting a table into multiple ones which must be linked each time a query is made requiring data from the split tables.

4.2.1 Features of Database Normalization

- **Elimination of Data Redundancy:** One of the main features of normalization is to eliminate the data redundancy that can occur in a database. Data redundancy refers to the repetition of data in different parts of the database. Normalization helps in reducing or eliminating this redundancy, which can improve the efficiency and consistency of the database.
- **Ensuring Data Consistency:** Normalization helps in ensuring that the data in the database is consistent and accurate. By eliminating redundancy, normalization helps in preventing inconsistencies and contradictions that can arise due to different versions of the same data.
- **Simplification of Data Management:** Normalization simplifies the process of managing data in a database. By breaking down a complex data structure into simpler tables, normalization makes it easier to manage the data, update it, and retrieve it.
- **Improved Database Design:** Normalization helps in improving the overall design of the database. By organizing the data in a structured and systematic way, normalization makes it easier to design and maintain the database. It also makes the database more flexible and adaptable to changing business needs.
- **Avoiding Update Anomalies:** Normalization helps in avoiding update anomalies, which can occur when updating a single record in a table affects multiple records in other tables. Normalization ensures that each table contains only one type of data and that the relationships between the tables are clearly defined, which helps in avoiding such anomalies.
- **Standardization:** Normalization helps in standardizing the data in the database. By organizing the data into tables and defining relationships between them, normalization helps in ensuring that the data is stored in a consistent and uniform manner.

4.2.2 Importance of Normalization

- Normalization is an important process in database design that helps improve the database's efficiency, consistency, and accuracy. It makes it easier to manage and maintain the data and ensures that the database is adaptable to changing business needs.

Database normalization is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data but at different places).

- Data redundancy unnecessarily increases the size of the database as the same data is repeated in many places. Inconsistency problems also arise during insert, delete, and update operations.
- In the relational model, there exist standard methods to quantify how efficient a database is. These methods are called normal forms and there are algorithms to convert a given database into normal forms.
- Normalization generally involves splitting a table into multiple ones which must be linked each time a query is made requiring data from the split tables.

4.2.3 Need For Normalization

QUESTIONS

1. Why there is need for normalization? (Dec. 23, 5 M)

- The main reason for normalizing the relations is removing these anomalies. Failure to eliminate anomalies leads to data redundancy and can cause data integrity and other problems as the database grows.
- Normalization consists of a series of guidelines that helps to guide you in creating a good database structure.

4.3 TYPES OF ANOMALIES

- If a database design is not perfect, it may contain anomalies, which are like a bad dream for any database administrator. Managing a database with anomalies is next to impossible.
- (a) **Problems without Normalization in DBMS**
 - If a table is not properly normalized and has data redundancy (repetition) then it will not only **eat up extra memory space** but will also make it difficult for you to handle and update the data in the database, without losing data.
 - Insertion, Updation, and Deletion Anomalies are very frequent if the database is not normalized.
 - To understand these anomalies let us take an example of a **Student** table.

Table 4.1

rollno	name	branch	hod	office_tel
401	Akon	CSE	Mr. X	53337
402	Bkon	CSE	Mr. X	53337
403	Ckon	CSE	Mr. X	53337
404	Dkon	CSE	Mr. X	53337

- In the table above, we have data for four Computer Sci. students.
- As we can see, data for the fields **branch**, **hod** (Head of Department), and **office_tel** are repeated for the students who are in the same branch in the college, this is **Data Redundancy**.

1. Insertion Anomaly in DBMS

- Suppose for a new admission, until and unless a student opts for a branch, data of the student cannot be inserted, or else we will have to set the branch information as **NULL**.
- Also, if we have to insert data for 100 students of the same branch, then the branch information will be repeated for all those 100 students.
- These scenarios are nothing but **Insertion anomalies**.
- If you have to repeat the same data in every row of data, it is better to **keep the data separately** and **reference that data** in each row.
- So in the above table, we can keep the branch information separately, and just use the **branch_id** in the student table, where **branch_id** can be used to get the branch information.

2. Updation Anomaly in DBMS

- What if Mr. X leaves the college? or Mr. X is no longer the HOD of the computer science department? In that case, all the student records will have to be updated, and if by mistake we miss any record, it will lead to data inconsistency.
- This is an Updation anomaly because you need to update all the records in your table just because one piece of information got changed.

3. Deletion Anomaly in DBMS

- In our **Student** table, two different pieces of information are kept together, the **Student information** and the **Branch information**.
- So if only a single student is enrolled in a branch, and that student leaves the college, or for some reason, the entry for the student is deleted, we will lose the branch information too.
- So never in DBMS, we should keep two different entities together, which in the above example is Student and branch.
- The solution for all the three anomalies described above is to keep the **student** information and the **branch** information in two different tables. And use the **branch_id** in the student table to reference the branch.

4.3.1 Primary Key and Non-key Attributes

- Before we move on to learn different Normal Forms in DBMS, let us first understand what is a primary key and what are non-key attributes.

Primary key		Non-key attributes	
student_id	student_name	mobile	gender
1	John	9797979797	Male
2	Ron	7878787878	Male
3	Pom	8282828282	Female

Fig. 4.1: Primary and Non-Primary Attributes

- In the table above, the **student_id** column is a **primary key** because using the **student_id** value we can uniquely identify each row of data, hence the *remaining columns* then become the **non-key attributes**.

4.3.2 Dependencies

- A dependency is a constraint that governs or defines the relationship between two or more attributes.
- In a database, it happens when information recorded in the same table uniquely determines other information stored in the same table.
- This may also be described as a relationship in which knowing the value of one attribute (or collection of attributes) in the same table tells you the value of another attribute (or set of attributes).
- It is critical to understand database dependencies since they serve as the foundation for database normalization.

4.4 TYPES OF DEPENDENCIES IN DBMS

- In DBMS, it has the following types:
 - Functional Dependency,
 - Fully-Functional Dependency,
 - Transitive Dependency,
 - Multivalued Dependency,
 - Partial Dependency.

4.4.1 Functional Dependencies

QUESTIONS

- Describe type of functional dependencies with suitable example. (May 19, 5 M)
 - Explain type of functional dependencies with suitable example. (Dec. 24, 5 M)
- A functional dependency (FD) is a relationship that exists between two attributes in a database, typically the primary key and additional non-key attributes.

Consider it a link between two qualities of the same relation.

- A dependency is denoted by an arrow " $\rightarrow$ ".
- If C determines D functionally, then $C \rightarrow D$.
- Functional dependency, indicated as $C \rightarrow D$, is a relationship between two sets of attributes, C and D. In this case, C is referred to as the "**determinant**", and D is referred to as the "**dependent**".
- Functional Dependency aids in the maintenance of database data quality.

(A) Functional Dependency Rules:

Inference Rules:

Axioms:

A relational database's functional dependencies can be inferred using Armstrong's axioms, a set of inference principles. Armstrong, William W., created them.

Functional Dependencies Axioms:

- The reflexive rule** states that if D is a subset of C, then D is determined by C, i.e. $C \rightarrow D$.
- The augmentation rule**, also known as the partial dependency rule, states that if D is determined by C, then CZ determines DZ for any Z. Every non-key attribute is required to be totally dependent on the Primary Key, according to it. i.e. If $C \rightarrow D$, then $CZ \rightarrow DZ$ for any Z.
- Transitivity rule** states that if D is determined by C and Z is determined by D, then C must also determine Z, i.e. if $C \rightarrow D$ and $D \rightarrow Z$, then $C \rightarrow Z$.

Decomposition:

It is a rule that stipulates that if a table appears to contain two entities determined by the same primary key, it should be split into two independent tables.

According to this rule, if C determines D and Z, C also determines D and Z individually. i.e. if $C \rightarrow DZ$ then $C \rightarrow D$ and $C \rightarrow Z$.

Union:

It suggests that if two tables are independent yet have the same Primary Key, they should be combined. It states that C must determine D and Z if C determines D and C determines Z.

i.e. if $C \rightarrow D$ and $C \rightarrow Z$ then $C \rightarrow DZ$.

(B) Terms:

- Dependent:** It is shown on the functional dependency diagram's right side.
- Determinant:** It is shown on the functional dependency Diagram's left side.
- Non-normalized table:** A table containing redundant data.

Examples

Example 1: Here, we have a table named **Student**.

<Student>

StuID	StuName	StuAge
E01	Rose	14
E02	Rolly	13

Here, **StuName** in the preceding table is functionally dependent on **StuID** since **StuName** can only accept one value for the specified value of **StuID**, i.e. Because a student's name can be uniquely determined from an ID, **StuName** can be considered to be dependent on **StuID**.

StuID → **StuName**

However, the converse assertion (**StuName** → **StuID**) is false because multiple students can have the same name but have different **StuID**'s.

Example 2: We have a table **Employee**.

<Employee>

Employee_No	E_Name	E_Salary	Address
1	Dolly	60000	Seoul
2	Flora	48000	BukchonHanok
3	Anni	35000	Seoul

(A) We can Deduce Several Valid Functional Dependencies from the Preceding Table:

- In this case, knowing the value of **Employee_No** allows us to access **E_Name**, **Address**, **E_Salary**, and so on. As a result, the **Address**, **E Name**, and **E Salary** are all functionally dependent on **Employee No**.

- Employee_No** → {**E_Name**, **E_Salary**, **Address**}: **Employee_No** can decide the values of fields **E_Name**, **E_Salary**, and **Address** in this case, resulting in a legal Functional dependence.

- Employee_No** → **E_Salary**, Because **Employee_No** can determine the entire set of {**E_Name**, **E_Salary**, and **Address**}, it can also determine its subset **E_Salary**.

- More valid functional dependents include: **Employee_No** → **name**, {**Employee_No**, **E_Name**} → {**E_Salary**, **Address**}, and so on.

Here are Some Invalid Functional Dependencies:

- E_Name** → **E_Salary**: This is not an acceptable functional dependency because employees with the same name can have different salaries.

- Address** → **E_Salary**: Different salaries can be given to the employees of the same **Address**; for example, **E_Salary** 60000 and 35000 in the preceding table belong to employees of the same address, "Seoul"; hence **Address** → **E_Salary** is an incorrect functional dependency.

- More invalid functional dependencies include: **E_Name** → **Employee_No**, {**E_Name**, **E_Salary**} → **Employee_No**, and so on.

(B) Types of Functional Dependencies

- Trivial Functional Dependency,
- Non-Trivial Functional Dependency,
- Multivalued Dependency,
- Transitive Dependency.

1. Trivial Functional Dependency:

- A "dependent" in Trivial functional dependency is always a subset of the "determinant".
- A functional dependency is said to be trivial if the attributes on its right side are a subset of the attributes on its left side.
- If D is a subset of C, C → D is referred to as a Trivial Functional Dependency.

Example: Take a look at the Student table below.

<Student>

Roll_No	S_Name	S_Age
1	John	13
2	Riya	12
3	Giya	15
4	Jolly	16

- {**Roll_No**, **S_Name**} → **S_Name** is a Trivial functional dependency in this case because the dependant **S_Name** is a subset of the determinant {**Roll_No**, **S_Name**}.
- { **Roll_No** } → { **Roll_No** }, { **S_Name** } → { **S_Name** } and { **S_Age** } → { **S_Age** } are also Trivial.

2. Non-Trivial Functional Dependency:

- It is the inverse of Trivial functional dependence. Formally, a Dependent is a Non-trivial functional dependency if it is not a subset of the determinant.
- If D is not a subset of C, C → D is said to have a non-trivial functional dependency. Non-trivial functional dependency is defined as a functional dependency C → D where, C is a set of attributes and D is also a set of attributes but not a subset of C.

Example: Consider the Student table below.

<Student>

Roll_No	S_Name	S_Age
1	John	13
2	Riya	12
3	Giya	15
4	Jolly	16

- Roll_No → S_Name is a non-trivial functional dependency in this case since S_Name (dependent) is not a subset of Roll_No (determinant).
- Similarly, {Roll_No, Name} → Age are non-trivial functional dependencies.
- Multivalued Functional Dependency.
- In multivalued functional dependency, attributes in the dependent set are not dependent on one another.
- For example, C {D, Z} is referred to as a Multivalued functional dependency if there is no functional dependency between D and Z.

Example: Take a look at the Student table below.

<Student>

Roll_No	S_Name	S_Age
1	John	13
2	Riya	12
3	Giya	15
4	Jolly	16

- {Roll_No} → {S_Name, S_Age} is a Multivalued functional dependency in this case because the "dependent values" S_Name and S_Age are not functionally dependent (i.e. S_Name → S_Age or S_Age → S_Name does not exist).

4. Transitive Functional Dependency:

- Consider two functional dependencies, C → D and D → Z; C → Z must exist according to the transitivity principle. This is referred to as a **Transitive Functional dependency**.
- In transitive functional dependency, the dependent is dependent on the determinant indirectly.

Example: Consider the Student table below.

<Student>

Roll_No	S_Name	S_Age	Street_No
1	John	AC	12
2	Riya	BH	11
3	Giya	MV	14
4	Jolly	CD	18

- Roll_No → S_Dept and S_Dept → Street_No are correct here. As a result, Roll_No → Street_No is a valid functional dependency, according to the principle of transitivity.

(C) Benefits of Functional Dependency

- Functional Dependency prevents data duplication. As a result, the same data does not appear several times in that database.
- It assists you in maintaining the database's data quality.
- It assists you in defining database semantics and constraints.
- It aids you in spotting flawed designs.
- It aids you in locating database design information.
- The Normalization method begins with identifying the potential keys in the relation. It is impossible to locate candidate keys and normalize the database without functional dependencies.

4.4.2 Fully Functional Dependency

- A functional dependency C → D, a fully functional dependency is one in which, if any attribute x from C is removed, the "dependency" no longer exists.
- If D is "fully functional dependent" on C, it is not functionally dependent on any of the valid subsets of C.
- i.e. Attribute Z in the relation CDE → Z is "fully functionally dependent" on CDE and not on any appropriate subset of CDE. That is, CDE subsets such as CD, DE, C, D, and so on cannot determine Z.
- Also;
 - Full Functional Dependency corresponds to the Second Normal Form normalization standard.
 - Functional dependency improves the data quality of our database.
 - In this dependency, the non-prime property is functionally reliant on the candidate key.
 - The full dependency on database attributes helps to assure data integrity and eliminate data abnormalities.

Example: Here, we have a table named Supply.

<Supply>

Seller_Id	Product_id	T_price
1	1	530
2	1	535
1	2	100
2	2	101
3	1	342

- According to the Table, neither Seller_id nor Product_id can uniquely determine the price, but both Seller_id and Product_id combined can.
- As a result, we can say that T_price is "fully functionally dependent" on Seller_id nor Product_id.
- This outlines and demonstrates our fully functional dependency:
 $\{ \text{Seller_id}, \text{Product_id} \} \rightarrow \text{T_Price}$

4.4.3 Partial Functional Dependency

- A functional dependency $C \rightarrow D$, If the dependency does hold after removing any attribute x from C, then it is said to be a **Partial Functional Dependency**.
- A functional dependency $C \rightarrow Y$, If D is functionally dependent on C and may be determined by any appropriate subset of C, there is a partial dependency.
- i.e. We have an $CF \rightarrow D$, $C \rightarrow E$, and $E \rightarrow D$ relation. Now, let us compute the closure of $\{C\}^+ = CED$. In this case, C can determine D on its own, implying that D is partially dependent on CF.

Also;

- In partial functional dependency, the non-prime attribute is functionally dependent on a component of a candidate key.
- The normalizing standard of the Second Normal Form does not apply to Partial Functional Dependency. 2NF, on the other hand, eliminates Partial Dependency.
- Partially dependent data does not improve data quality. It must be removed before normalization in the second normal form may occur.

(A) Cause of Partial Dependency:

- Partial dependency happens when a non-prime attribute is functionally dependent on a portion of the given candidate key, as we saw in the preceding section.
- In other words, partial dependency arises when an attribute in a table is dependent on only a portion of the primary key rather than the entire key.

Example: We have a table called **Student** here.

<Student>

Roll_No	S_Name	S_Course
1	John	DBMS
2	Riya	C++
3	Giya	Java
4	Jolly	C

- We can see that the attributes S_Name and Roll_No can both uniquely identify a S_Course. As a result, we might argue that the relationship is partly dependent.

4.4.4 Transitive Dependency

- A transitive dependence is any non-prime attribute other than the candidate key that is reliant on another non-prime attribute that is wholly dependent on the candidate key.
- Transitive Dependency occurs when an indirect interaction results in functional dependency. As a result, if $C \rightarrow D$ and $D \rightarrow Z$ are true, then $C \rightarrow Z$ is a transitive dependency.
- Transitive dependency causes deletion, update, and insertion errors in the database and is regarded as poor database design.
- To reach 3NF, one must first eliminate Transitive Dependency.

Note: Only when two Functional Dependencies establish an indirect functional dependency can it be transitive. As an example,

- When the following functional dependencies hold true, $C \rightarrow E$ is a transitive dependency:
 - $C \rightarrow D$
 - D does not imply C
 - $C \rightarrow E$
- Only in the case of some given relation of three or more attributes can transitive dependency occur effortlessly. Such a dependency aids us in normalizing the database in its 3rd Normal Form (3NF).

Example: Here, we have a table **Telecast_show**.

<Telecast_show>

Id_show	Id_telecast	Type_telecast	Cost_CD
F01	S01	Romantic	30
F02	S02	Thriller	50
F03	S03	Comedy	20

(Because of a transitive functional relationship, the table above is not in its 3NF.)

$\text{Id_show} \rightarrow \text{Id_telecast}$

$\text{Id_telecast} \rightarrow \text{Type_telecast}$

As a result, the following functional dependency is transitive.

$\text{Id_show} \rightarrow \text{Type_telecast}$

(A) Avoiding Transitive Functional Dependency

- According to the preceding statement, the relation <Telecast> violates the 3NF (3rd Normal Form).

- To address this violation, we must split the tables in order to remove the transitive functional relationship.

<show>

Id_show	Id_telecast	Cost_CD
F01	S01	30
F02	S02	50
F03	S03	20

<telecast>

Id_telecast	Type_telecast
S09	Thriller
S05	Romantic
S09	Comedy

The preceding relationship is now in the Third Normal Form (3NF) of Normalization.

4.4.5 Multivalued Dependency

- The term Multivalued Dependency refers to having several rows in a particular table. As a result, it implies that there are multiple other rows in the same table. A multivalued dependency would thus preclude the 4NF. Any multivalued dependency would involve at least three table attributes.
- When two separate attributes in a given table are independent of each other, multivalued dependency occurs. However, both of these are dependent on a third factor. At least two of the attributes are reliant on the third attribute in the multivalued dependence. This is why it always includes at least three of the qualities.

Example: Here we have a table **Car**.

<Car>

Model_car	Month_manu	Col_or
S2001	Jan	Yellow
S2002	Feb	Red
S2003	March	Yellow
S2004	April	Red
S2005	May	Yellow
S2006	June	Red

- In this scenario, the columns Col_or and Month_manu are both dependent on Model-car but independently of one another. As a result, we can refer to both of these columns as multivalued. Thus, they are dependent on Model_car. Here is a diagram of the dependencies we covered earlier:

Model_car $\twoheadrightarrow$ Month_manu

Model_car $\twoheadrightarrow$ Col_or

(A) Why Do We Use Multivalued Dependency in DBMS?

- When we face these two different ways, we always employ multivalued conditions:
 - When we wish to test the relationships or determine whether they are legal under certain arrangements of practical and multivalued dependencies.
 - When we want to know what restrictions exist on the arrangement of legal relationships; as a result, we will only be concerned with the relationships that fulfil a specific arrangement of practical and multivalued dependencies.

Occurrence:

- When two qualities in a table are independent of each other yet reliant on a third property, this is referred to as multivalued dependence.
- Because multivalued Dependency requires a minimum of two variables that are independent of each other in order to be dependent on the third variable, the minimum number of variables necessary is two.

(B) DBMS Dependency Conditions for Multivalued Dependency:

- If all of the following conditions are met, we can state that multivalued dependency exists.
- If any attribute 'C' has many dependencies on 'D,' for any relation R, for all the pair data values in table row R1 and table row R2, such that the relation

$$R1[C]=R2[C]$$

exists, and there is a relationship between row R3 and row R4 in the table such that

$$R1[C] = R2[C] = R3[C] = R4[C]$$

$$R1[D] = R3[D], R2[D] = R4[D]$$

Then we can assert the existence of Multivalued Dependency (MVD).

That is, in Rows R1, R2, R3, and R4,

R1[C], R2[C], R3[C], and R4[C] must all have the same value.

The value of R1[D] should be equal to R3[D], and the value of R2[D] should be equal to R4[D].

Example: Here, we have a table **Course**.

<Course>

Row_	Name_	Course_work_	Hobby_
R1	Ronit	Java	Dancing
R2	Ronit	Python	Singing
R3	Ronit	Java	Dancing
R4	Ronit	Python	Singing

Because we have distinct values of Course_work_ and Hobby_ for the same value of Name "Ronit," we have multivalued dependents on Name_.

Verification of <Course> table.

Let us now examine the condition of MVD (Multivalued Dependency) in our table.

Condition 1:

$$R1[C] = R2[C] = R3[C] = R4[C]$$

From the table;

$$R1[C] = R2[C] = R3[C] = R4[C] = \text{'Ronit'}$$

As a result, condition 1 appears to be met.

Condition 2:

$$R1[D] = R3[D], R2[D] = R4[D]$$

From the table;

$$R1[D] = R3[D] = \text{'Java'}, R2[D] = R4[D] = \text{'Python'}$$

As a result, condition 2 appears to be met as well.

Condition 3:

$$R1[E] = R4[E], R2[E] = R3[E]$$

We can draw a conclusion from the table.

$$R1[E] = R4[E] = \text{'Dancing'}, R2[E] = R3[E] = \text{'Singing'}$$

As a result, condition 3 is likewise satisfied, indicating that MVD occurs in the provided situation.

We have now;

$$C \twoheadrightarrow D$$

And from the table, we obtained the following;

$$\text{Name}_\rightarrow \text{Course_work}_\rightarrow$$

$$\text{And for } C \twoheadrightarrow E, \text{ we have}$$

$$\text{Name}_\rightarrow \text{Hobby}_\rightarrow$$

Finally, in the given table, we can conclude with the conditional relation as

$$\text{'Name}_\rightarrow \text{Course_work}_\rightarrow$$

$$\text{'Name}_\rightarrow \text{Hobby}_\rightarrow$$

4.5 NORMALIZATION

QUESTIONS

1. Explain 1NF, 2NF, 3NF with suitable example. (May 21, 10 Marks)
2. What is normalization? Explain 1NF, 2NF, 3NF with suitable example. (May 21, 10 Marks)
3. Explain concept of First Normal Form, Give example for same. (May 22, Dec. 23, May 25, 10 Marks)
4. Explain 1NF, 2NF, 3NF and BCNF with suitable example. (May 23, 10 Marks)
5. Define Normalization? Explain 1NF, 2NF, 3NF and BCNF with suitable example. (May 24, 10 Marks)

- Normalization in DBMS systematically decomposes tables to store each data item once, preventing redundancy/duplication, anomalies during inserts/updates/deletes, and inconsistencies across operations. This process enhances data integrity by enforcing dependencies through primary/foreign keys and structured relationships.
- Edgar F. Codd, creator of the relational model, introduced First Normal Form (1NF) and extended it to Second (2NF) and Third Normal Form (3NF). Later, Codd and Raymond F. Boyce refined it into Boyce-Codd Normal Form (BCNF), addressing remaining dependency issues for higher integrity.

(A) Key Benefits:

Following normalization rules achieves:

- **Redundancy elimination:** Duplicate data across rows or tables is removed.
- **Logical organization:** Data follows natural dependencies and hierarchies.
- **Improved integrity:** Constraints prevent invalid states via key-based links.
- **Easier maintenance:** Updates propagate reliably without widespread changes

	1NF	2NF	3NF	4NF	5NF
Decomposition of relation	R	R ₁₁ R ₁₂	R ₂₁ R ₂₂ R ₂₃	R ₃₁ R ₃₂ R ₃₃ R ₃₄	R ₄₁ R ₄₂ R ₄₃ R ₄₄ R ₄₅
Conditions	Eliminate repeating groups	Eliminate partial functional dependency	Eliminate transitive dependency	Eliminate multi-values dependency	Eliminate job dependency

Fig. 4.2: Types of Normalization

4.5.1 1-NF 2NF (First Normal Form)

- 1NF (First Normal Form) is a property of a relational database table that ensures the following:
 1. **Atomicity:** Each column contains atomic (indivisible) values, meaning no multiple values or sets of values are stored in a single column.
 2. **Uniqueness:** Each record (row) in the table must be unique, and there should be no duplicate rows.
 3. **No repeating groups:** Each column must contain unique, single values, and there should be no repeating groups or arrays within a column.

Example of a Table Not in 1NF: Consider the following table that stores information about students and the courses they are enrolled in:

Table 4.2 (a)

Student_ID	Student_Name	Courses
101	Alice	Math, Physics
102	Bob	Chemistry, Biology
103	Charlie	Maths, Chemistry

In this table

- The Courses column contains multiple values for each student (for example, Alice has "Math, Physics" in one column, and Bob has "Chemistry, Biology").
- This violates 1NF, because each column must contain atomic values, and the Courses column should not contain multiple values.

Converting the Table to 1NF

- To convert the table to 1NF, we must ensure that each column contains only atomic values. So, we will create a new row for each course a student is enrolled in.
- The table in 1NF will look like this:

Table 4.2 (b)

Student_ID	Student_Name	Courses
101	Alice	Math, Physics
102	Bob	Chemistry, Biology
102	Bob	Chemistry, Biology
103	Charlie	Maths, Chemistry
103	Charlie	Maths, Chemistry

Why This is Now in 1NF?

- The Courses column has been split into individual rows, ensuring each column holds only a single value per row.
- There are no multiple values in any column (atomicity is preserved).
- The table now satisfies 1NF, as each column contains atomic values and there are no repeating groups or sets of values.

Summary:

- The original table was not in 1NF because the Courses column contained multiple values for each student.
- After splitting the data, the table now follows 1NF, with each row representing a single atomic value for each student-course relationship.

4.5.2 2NF (Second Normal Form)

- 2NF (Second Normal Form) builds upon the rules of 1NF (First Normal Form) and addresses partial dependencies. A table is in 2NF if:
 - It is in 1NF.

- It does not have partial dependencies. That means every non-prime attribute (non-key attribute) must be fully dependent on the entire primary key, not just part of it.

- A partial dependency occurs when a non-key attribute depends only on part of a composite primary key rather than the whole key.
- Example of a Table Not in 2NF: Let us consider a table that stores information about student enrollments and courses:

Table 4.3

Student_ID	Course_ID	Student_Name	Course_Name	Instructor
101	301	Alice	Maths	Dr. Smith
101	302	Alice	Physics	Dr. Jones
102	301	Bob	Maths	Dr. Smith
103	303	Charlie	Chemistry	Dr. White

In this table:

- Primary Key:** The combination of Student_ID and Course_ID uniquely identifies each row because a student can take multiple courses, and each course can have multiple students.
- The non-key attributes are Student_Name, Course_Name, and Instructor.

Why This Table Is Not in 2NF?

- The table is in 1NF, as it contains atomic values.
- However, there are partial dependencies:
 - Student_Name depends only on Student_ID, not the whole composite key (Student_ID, Course_ID).
 - Course_Name and Instructor depend only on Course_ID, not the whole composite key (Student_ID, Course_ID).

Since these non-key attributes depend only on part of the primary key, the table violates 2NF.

Converting the Table to 2NF:

- To bring the table into 2NF, we need to remove partial dependencies. We can do this by creating separate tables for students, courses, and enrollments, where each non-key attribute is fully dependent on the primary key.

1. Student Table (Stores student-specific information):

Table 4.4

Student_ID	Student_Name
101	Alice
102	Bob
103	Charlie

2. Course Table (Stores course-specific information):

Table 4.5

Course_ID	Course_Name	Instructor
301	Maths	Dr. Smith
302	Physics	Dr. Jones
303	Chemistry	Dr. White

3. Enrollment Table (Stores which student is enrolled in which course):

Table 4.6

Student_ID	Course_ID
101	301
101	302
102	301
103	303

Why This Table Is Now in 2NF?

- Each table is in 1NF, as all values are atomic.
- In the **Student_Table**, **Student_Name** depends entirely on **Student_ID** (a single attribute primary key).
- In the **Course Table**, both **Course_Name** and **Instructor** depend entirely on **Course_ID** (a single attribute primary key).
- In the **Enrollment Table**, the **Student_ID** and **Course_ID** together act as the composite primary key, and there are no partial dependencies.
- Now, there are no non-key attributes that depend on only part of the primary key. Each non-key attribute is fully functionally dependent on the entire primary key, satisfying 2NF.

Summary:

- The original table was **not in 2NF** because there were partial dependencies (e.g., **Student_Name** depends only on **Student_ID**, and **Course_Name** depends only on **Course_ID**).
- After splitting the table into three tables, each non-key attribute is fully dependent on the whole primary key, and the database is now in 2NF.

4.5.3 3NF (Third Normal Form)

- 3NF (Third Normal Form) builds upon the rules of 2NF (Second Normal Form) and addresses transitive dependencies. A table is in 3NF if:

- It is in 2NF.
- It has no transitive dependencies. This means that non-prime attributes (attributes that are not part of the primary key) should not depend on other

non-prime attributes. Every non-prime attribute should depend only on the primary key and not on other non-prime attributes.

Example of a Table Not in 3NF: Let us consider a table storing information about students, the courses they are enrolled in, and their respective instructors:

Table 4.7

Student_ID	Course_ID	Student_Name	Course_Name	Instructor	Instructor_Phone
101	301	Alice	Maths	Dr. Smith	123-456-7890
101	302	Alice	Physics	Dr. Jones	987-654-3210
102	301	Bob	Maths	Dr. Smith	123-456-7890
103	303	Charlie	Chemistry	Dr. White	555-123-4567

Why This Table is Not in 3NF?

- The table is in 2NF because it satisfies both **1NF** and **2NF** (no partial dependencies).
- However, there is a **transitive dependency**:
 - Instructor_Phone** depends on **Instructor**, which is not a primary key but a non prime attribute.
 - Instructor** itself depends on **Course_ID** (since each course has a specific instructor), and therefore **Instructor_Phone** is transitively dependent on **Course_ID** through **Instructor**.

This creates redundancy because the **Instructor_Phone** number is repeated for every row where the same **Instructor** is teaching the course.

Converting the Table to 3NF:

- To convert the table to **3NF**, we need to remove the transitive dependency.
- This can be done by separating the **Instructor** and **Instructor_Phone** information into a new table, and linking it with the **Course** table.

1. Student Table (stores student information):

Table 4.8

Student_ID	Student_Name
101	Alice
102	Bob
103	Charlie

2. **Course Table (stores course information, including the instructor):**

Table 4.9

Course_ID	Course_Name	Instructor_ID
301	Maths	1
302	Physics	2
303	Chemistry	3

3. **Instructor Table (stores instructor information):**

Table 4.10

Instructor_ID	Instructor	Instructor_Phone
1	Dr. Smith	123-456-7890
2	Dr. Jones	987-654-3210
3	Dr. White	555-123-4567

4. **Enrollment Table (stores which students are enrolled in which courses):**

Table 4.11

Student_ID	Course_ID
101	301
101	302
102	301
102	303

Why This Table is Now in 3NF?

- The database is now in **2NF**, as we eliminated partial dependencies by separating the student, course, and instructor information into distinct tables.
- The table is in 3NF because there are no transitive dependencies:
 - Instructor_Phone** now depends directly on the **Instructor_ID** in the **Instructor Table**, which is the **primary key** for that table.
 - Instructor** depends on **Instructor_ID**, and **Instructor_Phone** depends on **Instructor_ID**, so no non-prime attribute depends on another **non-prime** attribute in any table.

In essence, we have removed the indirect relationship between **Instructor_Phone** and **Course_ID** that existed through **Instructor**.

Summary

- The original table was not in 3NF because it had a **transitive dependency** (**Instructor_Phone** → **Instructor** → **Course_ID**).
- After normalizing the database into four separate tables, the transitive dependency was eliminated,

ensuring that all non-key attributes depend directly on the primary key, and the database is now in **3NF**.

4.5.4 BCNF (Boyce-Codd Normal Form)

- BCNF (Boyce-Codd Normal Form)** is a higher level of normalization that builds upon **3NF (Third Normal Form)**.

- A table is in **BCNF** if:

- It is in **3NF**.
- For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey. A superkey is a set of attributes that uniquely identifies a record in a table. In other words, if a non-prime attribute depends on another attribute, that attribute must be a superkey.

- This means **BCNF** removes any remaining functional dependencies where non-prime attributes are dependent on non-superkey attributes, which might still be allowed in **3NF**.

Example of a Table Not in BCNF: Let us consider a table that stores information about students, courses, and instructors:

Table 4.12

Student_ID	Course_ID	Instructor_ID	Instructor_Name
101	301	1	Dr. Smith
102	301	1	Dr. Jones
103	302	2	Dr. Smith
104	303	3	Dr. White

Functional Dependencies:

- Student_ID, Course_ID** → **Instructor_ID, Instructor_Name** (A combination of **Student_ID** and **Course_ID** uniquely determines the instructor's details).
- Instructor_ID** → **Instructor_Name** (Each instructor has a unique name).

Why This Table is Not in BCNF?

- The table is in **3NF** because there are no transitive dependencies, and all non-key attributes depend on the entire primary key.
- However, there is a **functional dependency** where **Instructor_ID** → **Instructor_Name**, but **Instructor_ID** is not a **superkey** (since **Student_ID, Course_ID** is the primary key). This violates **BCNF**, as **Instructor_ID** is not a superkey, but it determines the **Instructor_Name**.

Converting the Table to BCNF:

- To make the table **BCNF compliant**, we need to separate the instructor information into its own table, ensuring that **Instructor_ID** is a superkey in its own table.
- We can split the data into two tables:

1. Instructor Table (stores instructor details):

Table 4.13 (a)

Instructor_ID	Instructor_Name
1	Dr. Smith
2	Dr. Jones
3	Dr. White

2. Enrollment Table (stores student-course relationships):

Table 4.13 (b)

Student_ID	Course_ID	Instructor_ID
101	301	1
102	301	1
103	302	2
104	303	3

Why This is Now in BCNF?

- The **Instructor Table** satisfies **BCNF** because **Instructor_ID** is now a superkey in that table.
- The **Enrollment Table** still has **Student_ID**, **Course_ID** as the primary key, and there is no violation of **BCNF** because no non-key attribute depends on anything other than the superkey.
- By separating the data into two tables, we eliminated the non-superkey dependency (where **Instructor_ID** $\rightarrow$ **Instructor_Name**), ensuring the database is now in **BCNF**.

Summary:

- The original table violated **BCNF** because **Instructor_ID** determined **Instructor_Name**, but **Instructor_ID** was not a superkey.
- After splitting the table into two tables—one for **Instructor** and one for **Enrollment**—we ensured that every non-trivial functional dependency is based on a superkey, thus bringing the database into **BCNF**.

4.5.5 4NF (Fourth Normal Form)

- It is a higher level of normalization that deals with multi-valued dependencies (MVDs). A table is in 4NF if:
- It is in **BCNF** (Boyce-Codd Normal Form).
- It has no **multi-valued dependencies**. A **multi-valued dependency** occurs when one attribute determines

multiple independent values for another attribute, and those values are not related to other attributes in the table.

- In other words, a table is in **4NF** if no non-trivial multi-valued dependency exists. A multi-valued dependency exists when a row in a table implies several independent facts, each of which should be represented by a separate row.

What is a Multi-Valued Dependency (MVD)?

- A **multi-valued dependency (MVD)** occurs when one attribute (or set of attributes) determines multiple values for another attribute, but these multiple values are **independent** of each other.
- This means that there is no relationship between the multiple values for an attribute, and they should be stored separately. Formally, if for a relation **RRR**, we have attributes **A, B, CA, B, CA, B, C**, then a multi-valued dependency **A $\twoheadrightarrow$ BA** to **BA $\twoheadrightarrow$ B** holds if for each value of **AAA**, there is a set of values for **BBB** that is independent of the values in other attributes like **CCC**.

Example of a Table Not in 4NF: Let us consider a table that stores information about students, their courses, and the hobbies they have:

Table 4.14

Student_ID	Course	Hobby
101	Math	Reading
101	Physic	Painting
101	Math	Cycling
102	Chemistry	Gardening
102	Biology	Reading
103	History	Painting

Functional Dependencies:

- Student_ID, Course $\rightarrow$** (Student information and course are linked).
- Student_ID $\rightarrow$** (Student details are dependent on the **Student_ID**).

Why This Table is Not in 4NF?

- This table contains a multi-valued dependency:
 - Student_ID $\twoheadrightarrow$ Hobby:** A student can have multiple hobbies, but the hobbies are independent of the courses the student is enrolled in.
 - Student_ID $\twoheadrightarrow$ Course:** A student can be enrolled in multiple courses, but the courses are independent of the hobbies.

Thus, the table violates 4NF because we are storing both the courses and hobbies for each student in the

same table, which results in redundancy. Specifically, for **Student_ID** 101, we have:

- Math → Reading, Painting, Cycling
- Physics → Reading, Painting, Cycling

Converting the Table to 4NF:

- To bring this table into **4NF**, we need to eliminate the multi-valued dependencies by creating separate tables for the independent facts (courses and hobbies). Each fact (course and hobby) should be stored in a separate table with a reference to the **Student_ID**.
- We split the original table into two tables:

1. **Student_Courses Table (stores which courses a student is enrolled in):**

Table 4.15

Student_ID	Course
101	Math
101	Physic
102	Chemistry
102	Biology
103	History

2. **Student_Hobbies Table (stores the hobbies of a student):**

Table 4.16

Student_ID	Hobby
101	Reading
101	Painting
101	Cycling
102	Gardening
102	Reading
103	Painting

Why These Tables are Now in 4NF?

- Each table is in **BCNF** because each non-key attribute is fully dependent on the primary key, and there are no non-trivial functional dependencies violating BCNF.
- Both tables are now in **4NF** because:
 - The multi-valued dependencies are eliminated. The **hobbies** of a student and the **courses** a student is enrolled in are stored independently in separate tables.
 - There is no redundancy caused by storing multiple independent values for a student in the same table.

Summary:

- The original table violated **4NF** because it had multi-valued dependencies: **Student_ID** →→ Hobby and **Student_ID** →→ Course, which led to redundancy.
- By splitting the table into two separate tables — one for **Student_Courses** and one for **Student_Hobbies** — we eliminated the multi-valued dependencies and brought the table into **4NF**.

4.5.6 5NF (Fifth Normal Form)

- It is also known as Projection-Join Normal Form (PJNF), deals with join dependencies. A table is in 5NF if:
 1. It is in **4NF** (Fourth Normal Form).
 2. It does not contain any **join dependencies** that are not implied by the candidate keys.
- A **join dependency** occurs when a table can be reconstructed (or decomposed) into multiple tables, but the decomposition involves losing data because the relationships between attributes are not clear.
- In simpler terms, **5NF** requires that all data be fully reconstructed using joins without losing any information. A table is in **5NF** when it cannot be decomposed into smaller tables without losing data or introducing redundancy.

What is a Join Dependency?

- A **join dependency** occurs when a table contains attributes that, when decomposed into separate tables, can still be joined back together without loss of information, but the decomposition does not necessarily follow from the candidate keys.
- A table that satisfies **5NF** does not have unnecessary join dependencies.

Example of a Table Not in 5NF: Consider a table storing information about students, courses, and the instructors of those courses:

Table 4.17

Student_ID	Course_ID	Instructor_ID
101	301	1
101	302	2
101	301	1
102	303	3
103	302	2

Functional Dependencies:

- **Student_ID, Course_ID** → **Instructor_ID**
(A student is enrolled in a specific course, which is taught by a specific instructor).

Why This Table is Not in 5NF?

- In this table, we have a **join dependency** because it can be decomposed into smaller tables, but this decomposition does not naturally follow from the candidate keys and may result in redundancy:

1. Student_Courses Table:

Table 4.18

Student_ID	Course_ID
101	301
101	302
102	301
103	303
103	302

2. Course_Instructors Table:

Table 4.19

Student_ID	Course_ID
101	1
101	2
102	1
103	2
103	3

- This decomposition introduces redundancy and loss of information since when the tables are joined back, we might encounter situations where the **Instructor_ID** is incorrectly paired with a **Student_ID** and **Course_ID** combination, which leads to **spurious tuples** (invalid combinations).

Converting the Table to 5NF:

- To make the table comply with 5NF, we need to break it down further to ensure there is no unnecessary decomposition and that no additional relationships are implied by the decomposition.
- A proper decomposition would involve separating the attributes that do not have any intrinsic relationship but are being forced into a single table, resulting in a **lossless decomposition**.
- We can decompose the original table into the following smaller tables that each represent a **fact** independently:

1. Student_Courses Table (records which students are enrolled in which courses):

Table 4.20

Student_ID	Course_ID
101	301
101	302
102	301
103	303
103	302

2. Course_Instructors Table (records which courses are taught by which instructors):

Table 4.21

Student_ID	Course_ID
101	1
101	2
102	1
103	2
103	3

Why This is Now in 5NF?

- Each of the smaller tables now represents a **single fact**, and we have eliminated any join **dependencies** that might have caused redundant data.
- These smaller tables can be **joined** back together (using **Student_ID** and **Course_ID**, or **Instructor_ID**) without creating redundant or spurious data.
- The decomposition is now lossless because each table contains only facts that are directly related, and we can always reconstruct the original table by joining the smaller tables back together.

Summary:

- The original table violated **5NF** because it had a join dependency: it could be decomposed into smaller tables, but this decomposition did not naturally follow from the candidate keys and led to redundant data.
- After decomposing the table into three smaller tables (**Student_Courses**, **Course_Instructors**, and **Student_Instructors**), the table is now in 5NF because the decomposition is lossless, and no further splitting is possible without losing data or creating redundancy.

4.6 DECOMPOSITION RULES AND EXAMPLE

QUESTIONS

- Write short note on Lossless Decomposition.

(Dec. 19, Dec. 21, May 23, 5 M)

- Decomposition in the context of database normalization refers to breaking down a large, complex table into smaller, more manageable tables while preserving the original data and ensuring that no information is lost in the process.
- The goal is to achieve the desired normal form (e.g., 3NF, BCNF, 4NF, 5NF) through decomposition.

- There are certain rules that guide the decomposition of a table to achieve a higher normal form, particularly for 3NF, BCNF, and 4NF.

Decomposition Rules:

1. Lossless Decomposition:

- A decomposition is lossless if you can recover the original relation by joining the decomposed relations.
- This means no information is lost, and no spurious tuples are introduced when performing the join.

2. Dependency-Preserving Decomposition:

A decomposition is dependency-preserving if the functional dependencies that were valid in the original table still hold in the decomposed tables, so you don't lose the ability to enforce constraints.

3. Preserving Keys:

The keys of the original table should be preserved in the decomposed tables, so that they can still be used to uniquely identify records.

Example of Decomposition: Let us consider a table that contains information about students, courses, and instructors:

Table 4.22

Student_ID	Course_ID	Instructor_Name	Instructor_Phone	Student_Name
101	301	Dr. Smith	123-456-7890	Alics
102	301	Dr. Smith	123-456-7890	Bob
103	302	Dr. Jones	987-654-3210	Alice
104	303	Dr. White	555-123-4567	Charlie

Step 1: Identify Functional Dependencies

Based on the data, we can identify some functional dependencies:

- $Student_ID, Course_ID \rightarrow Student_Name$: A student has a specific name.
- $Course_ID \rightarrow Instructor$: A course has a specific instructor.
- $Instructor \rightarrow Instructor_Phone$: Each instructor has a specific phone number.
- $Student_ID, Course_ID \rightarrow Instructor$: A student takes a course with a specific instructor.

Step 2: Analyze Violations of Normal Forms

- 1NF:** The table is in 1NF as it does not contain any repeating groups or nested data.
- 2NF:** The table violates 2NF because it has partial dependencies. For example, $Instructor_Phone$ is dependent on $Instructor$, but $Instructor$ is not part of the primary key (which is $Student_ID, Course_ID$).

- 3NF:** The table violates 3NF because of the transitive dependency: $Instructor \rightarrow Instructor_Phone$ (non-prime attribute depends on non-prime attribute $Instructor$).

- BCNF:** The table violates BCNF because $Instructor \rightarrow Instructor_Phone$ and $Instructor$ is not a superkey.

Step 3: Decompose into 3NF (or BCNF) Using Decomposition Rules

To bring the table to 3NF, we will break it down into smaller tables:

- Student_Courses Table (Removes partial dependency):**

Table 4.23

Student_ID	Course_ID	Student_Name
101	301	Alics
102	301	Bob
101	302	Alice
103	303	Charlie

- Primary key:** $Student_ID, Course_ID$
- Functional dependency:** $Student_ID, Course_ID \rightarrow Student_Name$ (no partial dependencies)

- Course_Instructors Table (Removes transitive dependency):**

Table 4.24

Course_ID	Instructor_Name	Instructor_Phone
301	Dr. Smith	123-456-7890
302	Dr. Jones	987-654-3210
303	Dr. White	555-123-4567

- Primary key:** $Course_ID$
 - Functional dependency:** $Course_ID \rightarrow Instructor, Instructor \rightarrow Instructor_Phone$
- Instructor Table (Store information about instructors separately):**

Table 4.25

Instructor_Name	Instructor_Phone
Dr. Smith	123-456-7890
Dr. Jones	987-654-3210
Dr. White	555-123-4567

Step 4: Verify Decomposition

- Lossless Join:** We can join the three tables back together on the appropriate keys ($Student_ID, Course_ID$) to reconstruct the original table without losing any data.

- **Dependency Preservation:** The functional dependencies are preserved in the decomposed tables.
- For example, $\text{Course_ID} \rightarrow \text{Instructor}$ is directly represented in the **Course_Instructors Table**, and $\text{Instructor} \rightarrow \text{Instructor_Phone}$ is represented in the **Instructor Table**.
- **No Redundancy:** Each table now stores related information, and there is no unnecessary duplication of data. **Instructor_Phone** is stored only in the **Instructor Table**, and **Student_Name** is stored only in the **Student_Courses Table**.

1. Student_Courses Table:

Table 4.26

Student_ID	Course_ID	Student_Name
101	301	Alics
102	301	Bob
101	302	Alice
103	303	Charlie

Appointment_UNF

Table 4.29

Row	PatID	PatName	Phones	Diseases	DocNames	DocSpeciality	VisitDate
1	P01	Asha	9876, 9123	Fever, Cough	Dr. Rao, Dr. Mehta	Physician, Pulmonologist	2025-01-01
2	P02	Bharat	9988	Diabetes	Dr. Sen	Endocrinologist	2025-01-02
3	P03	Chitra	9090, 9191	Asthma	Dr. Mehta	Pulmonologist	2025-01-03
4	P04	Deepak	9898	Fever, Headache	Dr. Rao	Physician	2025-01-04
5	P05	Esha	9001	Migraine, Anxiety	Dr. Kapoor, Dr. Shah	Neurologist, Psychiatrist	2025-01-05
6	P06	Farhan	9111, 9222	Fever	Dr. Rao	Physician	2025-01-06
7	P07	Gita	9333	Asthma, Allergy	Dr. Mehta, Dr. Verma	Pulmonologist, Dermatologist	2025-01-07

Why this table is not in 1NF (issues)?

- **Non-atomic attributes:**
 - Phones stores multiple numbers (for example, "9876, 9123").
 - Diseases stores multiple diseases (for example, "Fever, Cough").
 - DocNames, DocSpeciality also contain lists.
- **Repeating groups:** multiple doctors and diseases within single row.

2. Course_Instructors Table:

Table 4.27

Course_ID	Instructor_Name	Instructor_Phone
301	Dr. Smith	123-456-7890
302	Dr. Jones	987-654-3210
303	Dr.White	555-123-4567

3. Instructor Table:

Table 4.28

Instructor_Name	Instructor_Phone
Dr. Smith	123-456-7890
Dr. Jones	987-654-3210
Dr.White	555-123-4567

Example: Consider a **Hospital Appointment System** where all data is first stored in a single unnormalized table.

Unnormalized table (UNF)

Problem statement: Hospital stores patient details, multiple phone numbers, multiple diseases, and multiple doctors in a *single* table called Appointment_UNF.

- **Insertion/Update anomalies:** changing a doctor's speciality or adding a new phone forces editing embedded lists.

Hence it **violates 1NF** which requires each cell to contain a single atomic value and no repeating groups.

Convert to 1NF (single-table version)

Strategy:

- One row per combination of (**PatID, Disease, Doctor**).
- Phones still tied here for clarity (we'll separate later).

Appointment_1NF

Table 4.30

PatID	PatName	Phone	Disease	DocName	DocSpeciality	VisitDate
P01	Asha	9876	Fever	Dr. Rao	Physician	2025-01-01
P01	Asha	9123	Cough	Dr. Mehta	Pulmonologist	2025-01-01
P02	Bharat	9988	Diabetes	Dr. Sen	Endocrinologist	2025-01-02
P03	Chitra	9090	Asthma	Dr. Mehta	Pulmonologist	2025-01-03
P03	Chitra	9191	Asthma	Dr. Mehta	Pulmonologist	2025-01-03
P04	Deepak	9898	Fever	Dr. Rao	Physician	2025-01-04
P04	Deepak	9898	Headache	Dr. Rao	Physician	2025-01-04
P05	Esha	9001	Migraine	Dr. Kapoor	Neurologist	2025-01-05
P05	Esha	9001	Anxiety	Dr. Shah	Psychiatrist	2025-01-05
P06	Farhan	9111	Fever	Dr. Rao	Physician	2025-01-06
P06	Farhan	9222	Fever	Dr. Rao	Physician	2025-01-06
P07	Gita	9333	Asthma	Dr. Mehta	Pulmonologist	2025-01-07
P07	Gita	9333	Allergy	Dr. Verma	Dermatologist	2025-01-07

Why Appointment_1NF is in 1NF?

- Every column has **atomic values** (no comma-separated lists).
- No repeating groups; each tuple is one patient-disease-doctor-phone combination.

Issues with Appointment_1NF (not in 2NF)

- Choose **composite primary key**:
PK = {PatID, Disease, DocName}
(one row per patient-disease-doctor).

Issues:

- Partial dependencies** (violate 2NF):
PatName, Phone, VisitDate depend only on **PatID**, not on Disease/DocName.
PatID → PatName, Phone, VisitDate
- DocSpeciality depends only on **DocName**, not on the full composite key.
DocName → DocSpeciality
- Massive redundancy**: changing a doctor's speciality needs multiple row updates; same for patient name.

Dependencies in Appointment_1NF

Functional dependencies:

- PatID → PatName, Phone, VisitDate (VisitDate per visit; for simplicity assume last visit date)
- PatID → Phone (really multivalued)
- DocName → DocSpeciality
- (PatID, Disease, DocName) → (entire row: assumed main key)

- Partial dependencies because non-key attributes depend on only part of the composite key → not 2NF.

Convert to 2NF – tables with data

- Decompose to remove partial dependencies.

Table 4.31: Patient Table

PatID (PK)	PatName	VisitDate
P01	Asha	2025-01-01
P02	Bharat	2025-01-02
P03	Chitra	2025-01-03
P04	Deepak	2025-01-04
P05	Esha	2025-01-05
P06	Farhan	2025-01-06
P07	Gita	2025-01-07

Table 4.32: Patient Phone Table

PatID (PK, FK)	Phone (PK)
P01	9876
P01	9123
P02	9988
P03	9090
P03	9191
P04	9898
P05	9001
P06	9111
P06	9222
P07	9333

Table 4.33: Doctor Table

DocName (PK)	DocSpeciality
Dr. Rao	Physician
Dr. Mehta	Pulmonologist
Dr. Sen	Endocrinologist
Dr. Kapoor	Neurologist
Dr. Shah	Psychiatrist
Dr. Verma	Dermatologist

Table 4.34: PatientDiseaseDoctor (Core Relationship)

PatID (PK, FK)	Disease (PK)	DocName (PK, FK)
P01	Fever	Dr. Rao
P01	Cough	Dr. Mehta
P02	Diabetes	Dr. Sen
P03	Asthma	Dr. Mehta
P04	Fever	Dr. Rao
P04	Headache	Dr. Rao
P05	Migraine	Dr. Kapoor
P05	Anxiety	Dr. Shah
P06	Fever	Dr. Rao
P07	Asthma	Dr. Mehta
P07	Allergy	Dr. Verma

How dependencies are resolved (2NF)?

- All attributes depending only on **PatID** moved to Patient and PatientPhone.
- All attributes depending only on **DocName** moved to Doctor.
- The relationship table PatientDiseaseDoctor contains only attributes that depend on the **entire composite key** (PatID, Disease, DocName).
- Now each non-key attribute in every table depends on the *whole* key → tables are in 2NF.

3NF check: issues and why not yet 3NF (if any)

Examine each 2NF table:

Patient:

- **PK:** PatID; non-key attributes: PatName, VisitDate.
- Assume no further dependency like PatName → VisitDate; all directly depend on PatID.

Doctor:

- **PK:** DocName; DocSpeciality fully depends on DocName.

PatientPhone:

- **PK:** (PatID, Phone); no non-key attributes.

PatientDiseaseDoctor:

- **PK:** (PatID, Disease, DocName); no non-key attributes.

There is **no transitive dependency** (non-key → non-key) in any table, so they already satisfy **3NF**.

If exam expects explicit statement:

- No attribute in any table depends on another non-key attribute; all depend directly on a key.
- Hence **no 3NF violation**.

Dependencies and Conversion to 3NF (explicit)

Functional dependencies now:

- **Patient:** PatID → PatName, VisitDate
- **PatientPhone:** (PatID, Phone) → (no extra attributes)
- **Doctor:** DocName → DocSpeciality
- **PatientDiseaseDoctor:** (PatID, Disease, DocName) → (no extra attributes)

No transitive dependency of the form Key → non-key1 → non-key2.

Thus the decomposition from step 7 already gives **3NF** tables, no further splitting needed.

BCNF: Reasons, Issues, Dependencies, Conversion

- **BCNF condition:** For every non-trivial FD $X \rightarrow Y$, X must be a super key.

Check each table:

Patient:

- PatID → PatName, VisitDate; PatID is key → BCNF.

PatientPhone:

- Key is (PatID, Phone); no other FDs → BCNF.

Doctor:

- DocName → DocSpeciality; DocName is key → BCNF.

PatientDiseaseDoctor:

- Key is (PatID, Disease, DocName); no other FDs → BCNF.

So:

- **Why in BCNF:** all determinants are candidate keys.
- **Issues:** none; no FD with a non-key determinant.
- **Dependencies explanation:** listed above; all satisfy BCNF condition.
- **Conversion:** already achieved in previous decompositions; no additional tables needed.

Thus the schema is already in **BCNF**.

4NF: Reasons, Issues, Dependencies, Conversion

- 4NF handles **Multi-Valued Dependencies (MVDs)**.
- Earlier (in 1NF), there were independent MVDs:
 - $PatID \rightarrow Phone$
 - $PatID \rightarrow Disease$
 - $PatID \rightarrow Docname$

In final design:

- **PatientPhone:** Represents $PatID \rightarrow Phone$ with $(PatID, Phone)$ and $PatID$ is key.
- **PatientDiseaseDoctor:** Represents a specific 3-way relation; Not just an independent MVD.
- **Patient:** Has no multivalued attributes.
- **Doctor:** No MVDs.

There is **no table** where a single key determines two or more independent multivalued attributes simultaneously. Each MVD has been isolated in its own relation with key as determinant.

Therefore:

- **Why in 4NF:** no non-trivial MVD where determinant is not a super key.
- **Issues:** none remaining.
- **Dependencies:** MVDs are represented in separate tables (PatientPhone, etc.), satisfying 4NF.
- **Conversion:** effectively done when splitting phones/diseases/doctors out of the 1NF table.

5NF (PJNF): Reasons, Issues, Dependencies, Conversion

- 5NF (Project-Join Normal Form) deals with **join dependencies**: a relation can be decomposed into several projections such that it is equal to the join of those projections, and no further non-trivial JD exists that is not implied by candidate keys.

In the final schema:

Relations are already "minimal":

- **Entity tables:** Patient, Doctor.
- **Binary relationship tables:** PatientPhone, PatientDiseaseDoctor.

Attempting to further split.

For example, PatientDiseaseDoctor into:

- PatientDisease(PatID, Disease) and DiseaseDoctor (Disease, DocName) would introduce **spurious tuples** when joined.
- For example, obtaining a patient with a doctor via the same disease even if that particular pairing never occurred).

Hence:

- No relation can be decomposed further into smaller relations with a **lossless** join without introducing invalid combinations.

- All non-trivial join dependencies are implied by the candidate keys.

So:

- **Why in 5NF:** each relation's only join dependencies are key-based, and no further non-trivial, lossless decomposition is possible.
- **Issues not in 5NF:** none in final design.
- **Conversion:** achieved implicitly through step-by-step decomposition from UNF $\rightarrow$ 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF/BCNF/4NF.
- **Final reason for 5NF:** schema is dependency-preserving, lossless, and has no further non-trivial JDs $\rightarrow$ in 5NF.

4.7 UNIVERSITY SOLVED QUESTIONS

1. Explain 1NF, 2NF, 3NF with suitable example. **(May 21, 10 Marks)**
2. What is normalization? Explain 1NF, 2NF, 3NF with suitable example.
3. Explain concept of First Normal Form, Give example for same. **(May 22, Dec. 23, May 25, 10 Marks)**
4. Explain 1NF, 2NF, 3NF and BCNF with suitable example. **(May 23, 10 Marks)**
5. Define Normalization? Explain 1NF, 2NF, 3NF and BCNF with suitable example. **(May 24, 10 Marks)**

Solution:

- **Normalization** is the process of organizing data in a database to reduce redundancy, avoid data anomalies (insert, update, delete anomalies), and improve data integrity.
- It involves dividing large tables into smaller, well-structured tables and defining relationships between them using keys.

Definition:

- **Normalization** is a systematic process of organizing data in a database to reduce redundancy, eliminate undesirable characteristics like insertion, update, and deletion anomalies, and ensure data integrity by dividing data into smaller, well-structured tables and establishing proper relationships between them using keys.

First Normal Form (1NF):

- A table is in **1NF** if:
 - All attributes contain atomic (indivisible) values.
 - No repeating groups or multi-valued attributes.
 - Each record can be uniquely identified.

Example (Not in 1NF)**Table 4.35**

Roll_No	Name	Subjects
1	Amit	DBMS, OS
2	Ravi	CN, DBMS

Note: Subjects has multiple values.

Convert to 1NF**Table 4.36**

Roll_No	Name	Subject
1	Amit	DBMS
1	Amit	OS
2	Ravi	CN
2	Ravi	DBMS

Result: Now all values are atomic.

Second Normal Form (2NF):

- A table is in **2NF** if:
 - It is already in **1NF**
 - No partial dependency** (non-key attribute depends on only part of a composite primary key)

Example (Not in 2NF)**Table 4.37**

Student_ID	Course_ID	Student_Name	Course_Name
S1	C1	Amit	DBMS
S2	C2	Ravi	OS

Primary Key: (Student_ID, Course_ID)

- Student_Name depends only on Student_ID
- Course_Name depends only on Course_ID

Note: Partial dependency exists.

Convert to 2NF:**Table 4.38: Student Table**

Student_ID	Student_Name
S1	Amit
S2	Ravi

Table 4.39: Course Table

Course_ID	Course_Name
C1	DBMS
C2	OS

Table 4.40: Enrollment Table

Student_ID	Course_ID
------------	-----------

Result: No partial dependency.

Third Normal Form (3NF):

- A table is in **3NF** if:
 - It is already in **2NF**
 - No transitive dependency** (non-key attribute depends on another non-key attribute)

Example (Not in 3NF)

Emp_ID	Emp_Name	Dept_ID	Dept_Name
--------	----------	---------	-----------

- Dept_Name depends on Dept_ID
- Dept_ID depends on Emp_ID

Note: Transitive dependency exists.

Convert to 3NF:**Table 4.41: Employee Table**

Emp_ID	Emp_Name	Dept_ID
--------	----------	---------

Table 4.42: Department Table

Dept_ID	Dept_Name
---------	-----------

Result: Transitive dependency removed.

Boyce-Codd Normal Form (BCNF)

- A table is in **BCNF** if:
 - It is in **3NF**
 - For every functional dependency $X \rightarrow Y$, X must be a super key

Example (Not in BCNF):

Student	Subject	Teacher
---------	---------	---------

- Student, Subject $\rightarrow$ Teacher
- Teacher $\rightarrow$ Subject

Note: Teacher is not a super key.

Convert to BCNF:**Table 4.43 (a): Teacher_Subject Table**

Teacher	Subject
---------	---------

Table 4.43 (b): Student_Teacher Table

Student	Teacher
---------	---------

Result: Now every determinant is a super key.

- List all Functional dependencies satisfied by the relation. (May 23, 10 Marks)

Table 4.44

X	Y	Z
X1	Y1	Z1
X1	Y2	Z1
X2	Y2	Z1
X2	Y2	Z1